DỊCH VỤ QUYẾT ĐỊNH CHO AGENT LẬP TRÌNH

htrnguyen
AIAI Laboratory

aisq.science htrnguyen.id.vn

TÓM TẮT

Agent lập trình có thể sinh kế hoạch và bản sửa, nhưng nhiều lựa chọn xung quanh chỉ có một tập đáp án nhỏ, rõ ràng. Báo cáo khảo sát khi nào nên tách những lựa chọn đó thành dịch vụ quyết định, khi nào nên dùng code xác định và cách đưa sự không chắc chắn vào quy trình. Từ nghiên cứu và hướng dẫn kỹ thuật được chọn, báo cáo giới thiệu Laya MCP Server như một triển khai giúp nhiều agent truy cập quyết định có giới hạn qua giao diện chung. Nội dung gồm khung chọn tác vụ, hợp đồng tích hợp, phương pháp làm việc tái lập và phép quan sát nhỏ có cả ca đúng lẫn sai. Báo cáo không khẳng định có huấn luyện model mới, benchmark tổng quát, chất lượng ngang Jev hay chắc chắn tăng năng suất.

1 PHẠM VI VÀ PHƯƠNG PHÁP

Đây là khảo sát tường thuật có mục tiêu, không phải tổng quan hệ thống. Nguồn được chọn theo liên quan trực tiếp tới tổ chức agent, định tuyến model, từ chối dự đoán, chọn context và đánh giá. Nghiên cứu gốc/hướng dẫn chính thức được phân biệt với bài giải thích Jev và quan sát trên project. Ngày tra cứu: 04/10/2026.

Đối tượng đọc là kỹ sư muốn đưa một công cụ phán đoán có giới hạn vào quy trình coding hiện có. Nội dung trung lập về triển khai; điều kiện cài đặt cụ thể được giữ trong README.

Dịch vụ quyết định có ích khi cải thiện được một phần đo lường được của quy trình. Agent vẫn cần xác minh thay đổi sau đó.

2 NGHIÊN CỨU LIÊN QUAN

Chủ đề / nguồn	Điểm liên quan tới tích hợp
Tổ chức workflow [7]	Anthropic phân biệt luồng định trước với agent tự chọn hành động, khuyến nghị bắt đầu từ cách ghép đơn giản.
Phán đoán hữu hạn [1]	Bài Jev trình bày quyết định ngữ nghĩa như giao diện riêng bên cạnh model sinh nội dung.
Định tuyến được học [8]	RouteLLM dùng dữ liệu ưu tiên để học chọn giữa model mạnh và yếu; kết quả đánh đổi thuộc điều kiện thử nghiệm của nghiên cứu.
Dự đoán có từ chối [9]	SelectiveNet học đồng thời dự đoán và từ chối; gợi ý đo chất lượng trên phần được chấp nhận khi thay đổi độ bao phủ.
Chọn context [10]	Context engineering xem thông tin đưa vào model là tài nguyên cần quản lý, không đơn thuần là prompt dài hơn.
Đánh giá agent [11]	Cần xét kết quả tác vụ và bằng chứng thực thi; một câu trả lời có vẻ hợp lý chưa đủ.

Bảng 1: Các hướng nghiên cứu và tổng hợp

Các nguồn gợi ý xác định câu hỏi và giao diện trước khi quyết định trường hợp nào có thể tiếp tục mà không cần review thêm. Phép đánh giá sau đó cần bao quát cả quy trình. Những nguồn này tạo cơ sở khảo sát thiết kế, không chứng minh checkpoint Laya đang dùng chính xác.

Project không triển khai RouteLLM như router được học và không dùng kiến trúc SelectiveNet. Giá trị tham khảo là cách đánh giá đánh đổi định tuyến và hành vi từ chối, thay vì sao chép một con số hoặc ngưỡng nổi bật.

Kiểm tra schema đánh giá cấu trúc response. Đánh giá model đo độ đúng trên dữ liệu liên quan. Đánh giá workflow đo tác động đến kết quả tác vụ. Mỗi mức cần bằng chứng riêng.

3 ỨNG DỤNG

Ứng dụng	Câu hỏi hữu hạn	Bằng chứng trong báo cáo
Định tuyến hỗ trợ	Thông điệp thuộc nhóm vấn đề có sẵn nào?	Đã quan sát một ví dụ rõ ràng.
Phân loại log	Trích đoạn trực tiếp cho thấy lỗi nào?	Đã quan sát một ca rõ và một ca mơ hồ.
Lọc tài liệu truy xuất	Đoạn này có trả lời câu hỏi được cung cấp?	Mẫu tích hợp; chưa đánh giá tác vụ ở đây.
Chọn ứng viên	Lựa chọn được cung cấp nào khớp tiêu chí?	Mẫu tích hợp; chưa đánh giá tác vụ ở đây.
Ưu tiên review	Mục nào nên được người/agent xem trước?	Đề xuất hỗ trợ; không phải cổng phát hành.

Bảng 2: Ứng dụng và khung lựa chọn tác vụ

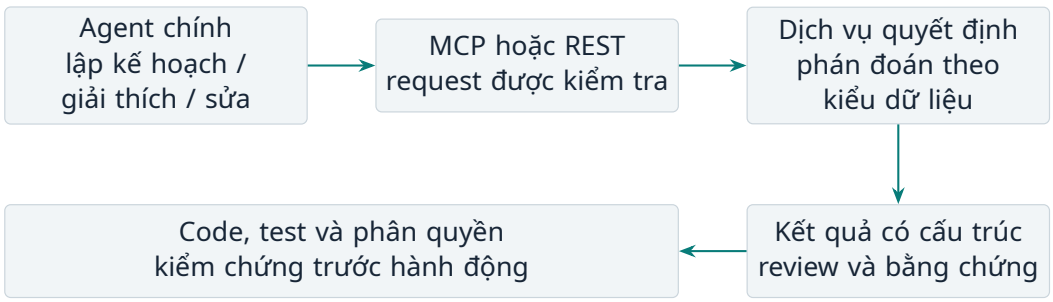
Đầu ra có biểu diễn bằng tập nhỏ, ổn định không? Người review có gán nhãn nhất quán cho mẫu đại diện không? Quyết định có đủ thường xuyên để bù chi phí phụ thuộc và review không? Nếu không, giữ logic trong agent chính hoặc code thông thường.

Parser nên nhận diện lỗi connection refused khi định dạng đã rõ. Phân loại ngữ nghĩa hợp lý hơn khi cùng biểu hiện được mô tả bằng nhiều cách tự nhiên. Dù vậy, nhãn chỉ là hướng điều tra, không phải nguyên nhân gốc đã xác nhận.

Không dùng service làm cổng duyệt duy nhất cho hành động phá hủy hoặc thay thế test. Luôn có nhánh xử lý bằng chứng chưa đủ.

4 TÍCH HỢP LAYA MCP

Laya MCP Server được xây dựng để cung cấp phán đoán hữu hạn nhất quán cho nhiều client coding, không nhúng riêng quy tắc project vào từng client. Tích hợp giữ giao diện chung, metadata rõ và hướng dẫn vận hành có thể đọc được. [2, 5, 6]



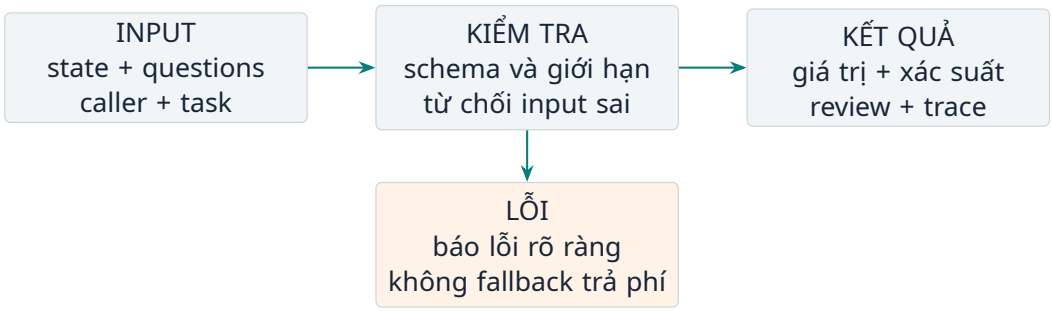
Hình 1: Kiến trúc tích hợp. Service dự đoán; agent và ứng dụng quyết định bước tiếp theo.

Nhu cầu thiết kế	Cách project đáp ứng
Truy cập dùng chung	Tool MCP và REST chuẩn hóa dùng cùng hợp đồng quyết định.
Cài đặt ít bước	Một lệnh setup đăng ký client global tìm thấy và skill dùng chung.
Tự khám phá cách dùng	Tool/resource hướng dẫn, tài liệu HTTP, Swagger, settings có thể copy.
Debug nhanh	Caller/task, trace ID, log có giới hạn và lỗi backend rõ ràng.

Bảng 3: Nhu cầu thiết kế và cách triển khai

Service dự đoán giá trị, không thi hành nó. Cài global giúp user cài đặt dùng tool xuyên project, không đồng nghĩa mọi user/máy đều được cấu hình. Người vận hành vẫn quản lý quyền truy cập và dữ liệu.

5 HỢP ĐỒNG QUYẾT ĐỊNH



Hình 2: Kiểm tra phân biệt request hợp lệ, kết quả dùng được và lỗi được báo rõ.

Kiểu	Mục đích	Giới hạn hiện tại
choice	Chọn một nhãn có sẵn	2-10 mô tả; thêm lựa chọn review khi cần.
noul	Ước lượng mệnh đề đúng/sai	Bản này luôn yêu cầu xem lại.
score	Chấm theo mô tả có thứ tự	2-5 mức; thử nghiệm, luôn review.

Bảng 4: Kiểu câu hỏi và giới hạn sử dụng

laya_decide và POST /v1/decide dùng cùng schema chuẩn hóa. Gửi state ngắn, một đến ba câu hỏi, caller và task. Mỗi câu hỏi có ID riêng, type và instructions; criteria phụ thuộc type.

Đầu ra	Cách đọc
value / probabilities	Dự đoán và phân bố của nó, không phải sự thật đã xác minh.
needs_review / warnings	Gợi ý chính sách và giới hạn. false không cho phép tự hành động.
usage / duration_ms / trace_id	Dữ liệu chẩn đoán request, không đánh giá tác vụ đã thành công.

Bảng 5: Các trường của response chuẩn hóa

Đầu ra chuẩn hóa sai cấu trúc hoặc bị cắt sẽ bị từ chối. Ngân sách câu hỏi hiện tại là 512 token; giữ bằng chứng và tiêu chí ngắn gọn. Confidence tóm tắt phân bố, không phải xác suất đáp án đúng. [4]

Với câu hỏi choice có K nhãn, gọi p_i là xác suất model trả cho nhãn i; $p_{(1)}$ và $p_{(2)}$ là hai giá trị lớn nhất. Nhãn dự đoán và độ chênh xác suất được tính như sau:

$$\hat{y} = \arg\max_{1 \le i \le K} p_i, \qquad m = p_{(1)} - p_{(2)}. \tag{1}$$

Triển khai hiện tại gắn cờ review cho câu hỏi choice khi

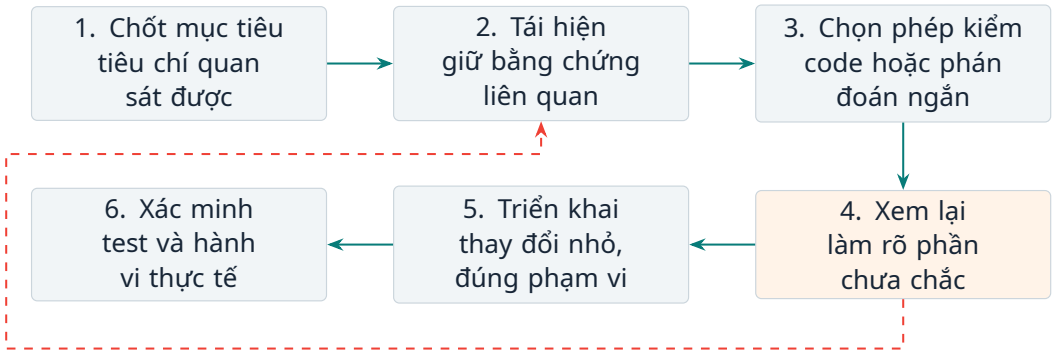
$$r = \mathbf{1}\left[p_{(1)} < 0.85 \ \vee \ m < 0.25 \ \vee \ \hat{y} \in \mathcal{U}\right], \tag{2}$$

trong đó $\mathcal{U}$ gồm các nhãn được so khớp không phân biệt hoa thường: `unknown`, `other`, `review` và `uncertain`. Đây là heuristic của triển khai, chưa phải quy tắc chấp nhận được hiệu chỉnh theo dữ liệu. Noul và score luôn được gắn cờ review trong bản này. Cờ review bằng false không cấp quyền thực hiện hành động.

Với ví dụ lỗi kết nối, $m = 0.9976 - 0.0014 = 0.9962$. Hai điều kiện số trong Công thức (2) đều không thỏa, nhãn chọn cũng không thuộc $\mathcal{U}$, nên service trả `needs_review=false`. Vẫn cần kiểm tra thực tế trước khi sửa hệ thống theo nhãn đó.

6 QUY TRÌNH LẬP TRÌNH

Tình huống: điều tra lỗi ứng dụng và sửa nhỏ nhất để hết lỗi mà không làm hỏng một luồng đang hợp lệ. Đây là phương pháp làm việc đề xuất, không phải chức năng service tự thực thi.



Hình 3: Quy trình coding đề xuất. Trường hợp chưa chắc quay lại thu bằng chứng.

Nêu input gây lỗi, hành vi mong muốn và phần không được ảnh hưởng. Yêu cầu agent đọc code chịu trách nhiệm trước khi sửa.

Giữ lệnh chính xác, output và log liên quan. Nếu test xác định đã chỉ ra điều kiện, dùng trực tiếp kết quả đó.

Chỉ dùng Laya cho phần cần hiểu ý nghĩa. Hỏi biểu hiện nào được log cho thấy, không hỏi nguyên nhân gốc khi thiếu bằng chứng.

Khi thiếu dữ kiện hoặc needs_review=true, thu thêm thông tin. Nhãn có vẻ hợp lý cũng chỉ dùng để ưu tiên bước kiểm tra tiếp theo.

Sửa trong phạm vi nhỏ, chạy lại ca lỗi và luồng không được ảnh hưởng, rồi báo cáo lệnh, kết quả và phần còn chưa chắc.

7 VÍ DỤ SỬ DỤNG

```
./setup.sh
```

Chạy trong checkout bằng user thường. Xem README của project để biết điều kiện cài đặt và triển khai. Mở phiên agent mới sau khi đăng ký global.

Điểm truy cập	Đường dẫn mặc định
Trang test / Swagger	http://127.0.0.1:30765/ và /docs
MCP / REST chuẩn hóa	/mcp và POST /v1/decide
Hướng dẫn cho agent	/llms.txt, /guide.md hoặc MCP laya_guide

Bảng 6: Các điểm truy cập service

```
{
  "state": "ConnectionError: database connection refused at localhost:5432.",
  "questions": [{
    "id": "category", "type": "choice",
    "instructions":
      "Classify only the failure directly shown by this log, not an unproven root cause.",
    "criteria": {
      "connectivity": "Cannot connect to a service",
      "application": "Application logic or assertion failure",
      "unknown": "Insufficient evidence"
    }
  }],
  "caller": "codex", "task": "debug-db-connection"
}
```

```
curl -fsS http://127.0.0.1:30765/v1/decide \
  -H 'Content-Type: application/json' \
  --data-binary @request.json
```

Lưu JSON thành request.json. Có thể gửi cùng các trường qua laya_decide hoặc Swagger. caller/task phục vụ truy vết, không xác thực danh tính.

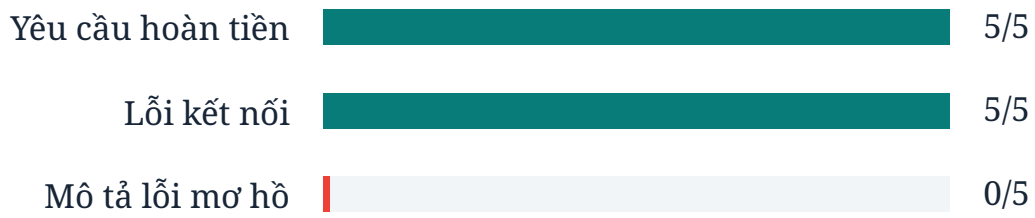
Phần tích hợp có thể dùng cờ review để chọn bước điều tra tiếp theo mà chưa thi hành hành động do model đề xuất:

```
decision = result["decisions"][0]
if decision["needs_review"]:
    next_step = "collect_more_evidence"
elif decision["value"] == "connectivity":
    next_step = "check_service_endpoint"
else:
    next_step = "inspect_application_log"
print(next_step)
```

Đoạn mã chỉ minh họa luồng điều khiển. Nó chưa xác minh service có hoạt động hay sửa ứng dụng. Với ca lỗi kết nối đã quan sát, bước tiếp theo là kiểm tra endpoint từ đúng môi trường nơi lỗi phát sinh.

8 KẾT QUẢ VÀ SAI SÓT

Phép quan sát ngày 04/10/2026 dùng ba mẫu do người viết chọn: mỗi mẫu một lượt warm-up và năm lượt đo. Nhãn kỳ vọng được chọn trước. Lặp lại không tạo thành mười lăm ca kiểm thử độc lập.



Hình 4: Số lượt trùng nhãn kỳ vọng trong năm lượt lặp; chỉ có ba mẫu riêng biệt.

Mẫu	Kỳ vọng / thực tế	Review
Yêu cầu hoàn tiền rõ ràng	billing / billing	false
Log connection refused	connectivity / connectivity	false
”Có lỗi rồi, giúp tôi xử lý.”	unknown / application	true

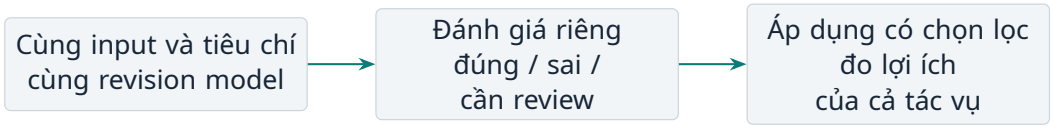
Bảng 7: Nhãn quan sát và cờ review

```
value: connectivity
probabilities: connectivity=0.9976, application=0.0014, unknown=0.001
margin: 0.9962
needs_review: false
warnings: [uncalibrated_for_your_task]
advisory: true
```

Nhãn lỗi kết nối phù hợp với log cung cấp. Nó không chứng minh database đã tắt. Cần kiểm tra listener, cấu hình endpoint và khả năng kết nối từ đúng môi trường thực thi trước khi quyết định sửa gì.

Mẫu mơ hồ cho P(application)=0,6364 và P(unknown)=0,0988. Review flag có ích trong ca này, nhưng chưa chứng minh rằng mọi dự đoán sai đều được gắn cờ.

9 ĐÁNH GIÁ



Hình 5: Quy trình đánh giá trước khi áp dụng dịch vụ quyết định.

```
Đọc laya_guide một lần khi cần dùng service.
Ưu tiên một câu hỏi choice ngắn, nhãn rõ ràng.
Truyền caller và mục đích task thật.
Xem kết quả là gợi ý; làm rõ phần chưa chắc.
Dùng code và test cho điều kiện chính xác.
Không thi hành chỉ vì model đồng ý.
Không retry vô hạn hoặc tự fallback trả phí.
```

So sánh tác vụ hoàn chỉnh có và không có lượt gọi quyết định bổ sung. Tính cả quyết định sai, công review, gọi lặp và thời gian khôi phục. Báo cáo này không khẳng định đã có phép so sánh năng suất end-to-end có kiểm soát.

Tạo tập mẫu có đáp án riêng cho tác vụ, gồm dữ kiện thiếu và trường hợp ngoài phạm vi. Tách mẫu đánh giá khỏi mẫu dùng để chỉnh câu hỏi. Giữ tiêu chí và revision model cố định khi so các phương án.

Service có log request ngắn gọn và chế độ debug JSON có giới hạn trong cấu hình triển khai. Dùng trace ID để nối request liên quan; không cộng thời gian request lồng nhau. Tránh đưa credential hoặc nội dung riêng tư vào dữ liệu test.

Giao diện tương thích không đồng nghĩa chất lượng model tương đương. Model card multilingual của Laya ghi nhận các giới hạn về hiệu chỉnh xác suất và chấm điểm. [3]

Cần đo hành vi review riêng với độ đúng của dự đoán. Đặt $a_i = 1$ nếu trường hợp i được tiếp tục mà không cần review thêm, và $a_i = 0$ nếu ngược lại. Độ bao phủ và tỷ lệ lỗi trên các trường hợp được chấp nhận là

$$C = \frac{\sum_{i=1}^{N} a_i}{N}, \qquad R = \frac{\sum_{i=1}^{N} a_i \, \mathbf{1}[\hat{y}_i \neq y_i]}{\sum_{i=1}^{N} a_i}. \tag{3}$$

R chỉ xác định khi có ít nhất một trường hợp được chấp nhận. Báo cáo cả hai đại lượng giúp thấy sự đánh đổi giữa chấp nhận nhiều trường hợp hơn và chấp nhận dự đoán sai; đây là vấn đề được nghiên cứu trong dự đoán có từ chối [9].

Nếu ba mẫu riêng biệt trong báo cáo được chấp nhận khi `needs_review=false`, độ bao phủ sẽ là $C = 2/3$ và tỷ lệ lỗi quan sát trên phần chấp nhận là $R = 0/2$. Mẫu còn lại được chuyển sang review. Các số đếm chỉ mô tả ba mẫu đó, không đủ để xác lập ngưỡng triển khai hay ước lượng độ chính xác chung.

10 GIỚI HẠN

Khảo sát tường thuật này hỗ trợ lựa chọn tích hợp nhưng chưa tạo thành benchmark tổng quát. Số mẫu quan sát quá ít để ước lượng độ chính xác. Nội dung và thiết kế riêng của báo cáo được ghi nhận cho AIAI Laboratory và htrnguyen; model, mã nguồn và công bố tham khảo giữ tác giả gốc.

TÀI LIỆU THAM KHẢO

[1] Avi Chawla (2026). Jev, Clearly Explained. https://blog.dailydoseofds.com/p/jev-clearly-explained.

[2] Convai Innovations. Laya: source and documentation. https://github.com/NandhaKishorM/laya.

[3] Convai Innovations. Laya multilingual model card. https://huggingface.co/convaiinnovations/laya-multilingual.

[4] TypeSafe AI. Confidence reference. https://docs.typesafe.ai/confidence.

[5] Model Context Protocol contributors. Protocol specification. https://modelcontextprotocol.io/specification/latest.

[6] AIAI Laboratory / htrnguyen. Laya MCP Server, v0.1.0. https://github.com/AIAI-Laboratory/laya-mcp-server.

[7] Anthropic (2024). Building effective agents. https://www.anthropic.com/engineering/building-effective-agents.

[8] Ong et al. (2024). RouteLLM: Learning to Route LLMs with Preference Data. https://arxiv.org/abs/2406.18665.

[9] Geifman & El-Yaniv (2019). SelectiveNet. ICML 2019. https://arxiv.org/abs/1901.09192.

[10] Anthropic (2025). Effective context engineering for AI agents. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents.

[11] Anthropic (2026). Demystifying evals for AI agents. https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents.